Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	192424104
Student Name	NAVEEN KUMAR SARAVANAN

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute 10 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform: ShopSphere is a growing e-commerce platform whose product catalogue currently holds n = 100,000 products and is expected to reach n = 1,000,000 within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

Module M1 – Duplicate-Listing Detector: Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n − 2 DO
   FOR j ← i + 1 TO n − 1 DO
      IF Product[i].title = Product[j].title AND Product[i].seller =
Product[j].seller THEN
         MarkDuplicate(j)
      END IF
   END FOR
END FOR
```

Module M2 – API Rate Limiter: The public API audits request windows of doubling size using the routine below. A team member claims that it is Θ(n log n) because the outer loop runs about log n times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
   FOR j ← 0 TO i − 1 DO
      Audit(Request[j])          // basic operation
   END FOR
   i ← i × 2
END WHILE
```

Module M3 – Product Search: Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by

comparing the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies T(n) = T(n/2) + 1, T(1) = 1.

Module M4 – Recommendation Engine: The engine splits the customer–product data into four parts of size n/2 (because of replicated partitions) and merges the partial results in linear time: T(n) = 4T(n/2) + n. A proposed redesign removes the replication, giving T(n) = 2T(n/2) + n.

Module M5 – Sales Analytics Engine: The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving T(n) = 2T(n/2) + n log n. Separately, profiling shows that the nightly report generator performs f(n) = 3n² + 10n log n + 50 operations.

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

(b) Analyse the Iterative Modules: For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's Θ(n log n) claim is correct and give the tight bound. Verify your M2 result for n = 1,024.

(c) Solve a Recurrence by Substitution: Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

(d) Apply the Master Theorem: For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a, b, f(n), n^(log_b a) and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to T(n) = 2T(n/2) + n log n; if not, derive the complexity using the extended Master Theorem or a recursion tree.

(e) Analyse Best, Average and Worst Cases: For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p, and evaluate it for n = 1,000,000 and p = 0.5.

(f) Prove an Asymptotic Bound: Using the formal definitions, prove that the report generator's cost f(n) = 3n² + 10n log n + 50 is Θ(n²) by finding constants c , c and n , and show that f(n) is not O(n).

(g) Scalability Decision: Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for n = 1,000,000. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach

Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Code of Conduct

I NAVEEN KUMAR SARAVANAN / 192424104 (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *Naveen Kumar Saravanan* Date: _____

Answer Sheet – Scalability Audit of ShopSphere

Conventions. log means $\log_2$. The servers execute **10 basic operations per second**, so time = operations ÷ 10 seconds. A 10-minute nightly window = 600 s = **6,000 operations**. Current catalogue n = 10^5; expected n = 10^6. For recurrences, n is a power of 2 and T(1) = 1 unless stated otherwise.

(a) Understand the Problem – input size, basic operation, expected growth

Module	Input size	Basic operation and where it occurs	Expected growth / justification
M1 Duplicate detector	n = number of products	The IF test comparing title and seller of Product[i] with Product[j]; executed in the innermost body of the nested loops.	Every pair (i, j) with i < j is compared, so about $n^2/2$ times → quadratic. Doubling n gives about 4× the work.
M2 Rate limiter	n = number of requests in the log	Audit(Request[j]) in the inner FOR loop.	Inner loop runs i = 1, 2, 4, … (not n) times, a geometric series dominated by its last term → linear. Doubling n gives about 2× the work.
M3-S1 Sequential search	n = catalogue size	Comparison of the search key with Product[k] during the scan.	Up to n comparisons → linear.
M3-S2 Binary search	n = index size	Comparison of the key with the middle entry; one per halving.	The range halves each step, so about log n + 1 comparisons → logarithmic. Doubling n adds only 1 comparison.
M4 Recommendation engine	n = size of the customer–product data	The linear-time merge step (the n work done at every node of the recursion tree).	Current T(n) = 4T(n/2) + n: the leaves (n^2 of them) dominate → quadratic. Redesign 2T(n/2) + n: equal work n on each of log n levels → n log n.
M5 Sales analytics	n = number of sales records	Record comparison inside the sort performed at every recursion level (cost n log n at the top).	Per level the sort cost shrinks from n log n; summing log n levels gives about n $\log^2 n$. Report generator f(n) = $3n^2$ + 10n log n + 50 is dominated by $3n^2$ → quadratic.

(b) Analyse the Iterative Modules

M1 – exact number of comparisons

For a fixed i the inner loop runs j = i+1 … n−1, i.e. (n − 1 − i) times. Summing over i = 0 … n−2:

$$C(n) = \sum_{i=0}^{n-2} (n - 1 - i) = (n-1) + (n-2) + \ldots + 2 + 1 = \sum_{k=1}^{n-1} k = \mathbf{n(n-1)/2}$$

The count does not depend on the data (no early exit), so best = average = worst case.

M1: C(n) = n(n−1)/2 = ½n^2 − ½n. Time = Θ(n^2). Space = Θ(1) extra (only i, j; the input itself is O(n)).

Meaning: n = 10^5 → 4,999,950,000 ≈ 5×10^9 comparisons; n = 10^6 → 499,999,500,000 ≈ 5×10^{11} comparisons (100× more for a 10× larger catalogue).

M2 – exact number of Audit operations

Let the outer loop variable take the values i = 2^t for t = 0, 1, 2, … while i < n. The last value satisfies 2^t < n, so t runs from 0 to k − 1 where k = ⌈$\log_2$ n⌉. The outer loop therefore executes k = ⌈log n⌉ times, and iteration t performs i = 2^t Audit calls:

$$A(n) = \sum_{t=0}^{k-1} 2^t = 2^k - 1 = \mathbf{2^{\lceil \log n \rceil} - 1}$$

Since n ≤ 2^k < 2n, we get **n − 1 ≤ A(n) ≤ 2n − 2**. For n a power of 2, A(n) = n − 1 exactly.

Is the Θ(n log n) claim correct? No. The team member bounded the inner loop by n in every iteration, but the inner loop runs only i times, and i is n only at the very end. The correct statement is the geometric series above: roughly n/2 + n/4 + … ≈ n. Θ(n log n) is a valid upper bound (O) but is not tight, so Θ(n log n) is wrong.

M2: A(n) = $2^{\lceil \log n \rceil}$ − 1, tight bound Θ(n) (between n−1 and 2n−2). Space = Θ(1).

Verification, n = 1,024 = 2^{10}: i takes 1, 2, 4, 8, 16, 32, 64, 128, 256, 512 (10 outer iterations, since 1024 < 1024 is false). Audit count = 1+2+4+...+512 = $2^{10} - 1$ = **1,023** = n − 1 ✓. The claimed n log n would predict 1,024 × 10 = 10,240, which is about 10× too large, confirming the bound is Θ(n).

(c) Recurrence by Backward Substitution – search version S2

T(n) = T(n/2) + 1, T(1) = 1. Take n = 2^k.

T(n) = T(n/2) + 1
= [T(n/4) + 1] + 1 = T(n/2^2) + 2
= [T(n/8) + 1] + 2 = T(n/2^3) + 3
…
= T(n/2^i) + i (pattern after i substitutions)

The recursion stops when n/2^i = 1, i.e. i = k = $\log_2$ n:

T(n) = T(1) + $\log_2$ n = 1 + $\log_2$ n

Check: T(1)=1 ✓, T(2)=T(1)+1=2=log 2+1 ✓, T(8)=4=log 8+1 ✓. For general n the closed form is ⌊$\log_2$ n⌋ + 1.

T(n) = $\log_2$ n + 1 (n a power of 2) ⇒ T(n) = Θ(log n).

Implementation	Time complexity	Space complexity
Iterative binary search (low, high, mid variables)	Θ(log n) worst/average; O(1) best	Θ(1) – a fixed number of variables
Recursive binary search	Θ(log n) worst/average; O(1) best	Θ(log n) – recursion stack depth ⌊log n⌋ + 1 frames (Θ(1) only if the compiler optimises tail calls)

Meaning: for n = 10^6 only ⌊19.93⌋ + 1 = 20 comparisons are needed; growing the catalogue 10× adds about 3–4 comparisons.

(d) Master Theorem

T(n) = aT(n/b) + f(n). Compare f(n) with $n^{\log_b a}$: Case 1 if f(n) = O($n^{\log_b a - \varepsilon}$) → Θ($n^{\log_b a}$); Case 2 if f(n) = Θ($n^{\log_b a}$) → Θ($n^{\log_b a}$ log n); Case 3 if f(n) = Ω($n^{\log_b a + \varepsilon}$) and the regularity condition holds → Θ(f(n)).

	M4 current: T(n)=4T(n/2)+n	M4 redesign: T(n)=2T(n/2)+n
a, b	a = 4, b = 2	a = 2, b = 2
f(n)	n	n
$n^{\log_b a}$	$n^{\log_2 4} = n^2$	$n^{\log_2 2} = n$
Comparison	f(n) = n = O($n^{2-\varepsilon}$) with ε = 1	f(n) = n = Θ(n)
Case	Case 1	Case 2
Result	**Θ(n^2)**	**Θ(n log n)**

When the data doubles: current design T(2n) ≈ 4·T(n) – the time **quadruples** (e.g. exact solution T(n) = $2n^2$ − n). Redesign: T(2n) = 2n·log(2n) = 2·T(n)·(1 + 1/log n) – the time slightly more than **doubles** (about 2.10× at n = 10^6). Removing the replication cuts the work from n^2 to n log n.

M5: T(n) = 2T(n/2) + n log n

Here a = 2, b = 2, $n^{\log_2 2} = n$, f(n) = n log n.

- Case 1: needs f(n) = O($n^{1-\varepsilon}$); n log n grows faster than n, so **no**.
- Case 2 (basic form): needs f(n) = Θ(n); n log n is not Θ(n), so **no**.
- Case 3: needs f(n) = Ω($n^{1+\varepsilon}$) for some ε > 0; but log n = o(n^{ε}) for every ε > 0, so n log n is not polynomially larger than n, **no**. f(n) falls in the gap between Cases 2 and 3.

Extended Master Theorem: if f(n) = Θ($n^{\log_b a}$ $\log^k$ n) with k ≥ 0, then T(n) = Θ($n^{\log_b a}$ $\log^{k+1}$ n). Here k = 1, so T(n) = Θ(n $\log^2$ n).

Recursion-tree verification: level i has 2^i nodes, each costing (n/2^i) log(n/2^i), so the level costs n(log n − i). With log n levels (i = 0 … log n − 1) plus n for the leaves:

$T(n) = n + \Sigma_{i=0}^{\log n - 1} n(\log n - i) = n + n \cdot \Sigma_{i=1}^{\log n} j = n + n \cdot \log n \cdot (\log n + 1)/2$

M5: no standard case applies; $T(n) = \Theta(n \log^2 n)$ (exactly $n + n \cdot \log n(\log n+1)/2$ for $n = 2^k$).

Separately, $f(n) = 3n^2 + 10n \log n + 50$ is quadratic (proved in part f).

(e) Best, Average and Worst Cases

Version	Best case	Average case	Worst case
S1 sequential	Key at position 1 → **1** comparison	Key equally likely at any position: (n+1)/2 if found; n if absent. Combined: see formula below → Θ(n)	Key at the last position or absent → **n** comparisons
S2 binary	Key is the middle entry → **1** comparison	About $\log_2 n - 1$ comparisons if found (about log n if absent) → Θ(log n)	Key absent, or found at the deepest level → **$\lfloor \log_2 n \rfloor + 1$** comparisons

S1 average with probability p that the product exists (if present, each of the n positions equally likely, probability p/n each, costing k comparisons for position k; if absent, probability 1 − p, costing n):

$A(n, p) = \Sigma_{k=1}^{n} k \cdot (p/n) + n(1 - p) = (p/n) \cdot n(n+1)/2 + n(1 - p) = p(n+1)/2 + n(1 - p)$

$A(n, p) = [\ (2 - p) \cdot n + p\ ] / 2$

Evaluate n = 1,000,000 and p = 0.5:

$A = 0.5 \times (1{,}000{,}001)/2 + 1{,}000{,}000 \times 0.5 = 250{,}000.25 + 500{,}000 =$ **750,000.25 comparisons**

(Check with the compact form: $(1.5 \times 10^6 + 0.5)/2 = 750{,}000.25$ ✓.) For S2 on $n = 10^6$ the worst case is 20 comparisons, so S2 is about 37,500× cheaper than the S1 average.

(f) Prove $f(n) = 3n^2 + 10n \log n + 50$ is $\Theta(n^2)$

Definition: $f(n) = \Theta(g(n))$ iff there exist constants $c_1, c_2 > 0$ and n_0 such that $c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n)$ for all $n \ge n_0$. Here $g(n) = n^2$.

Lower bound: since $10n \log n \ge 0$ and $50 > 0$ for $n \ge 1$, $f(n) \ge 3n^2$. So $c_1 = 3$ works for all $n \ge 1$.

Upper bound: for $n \ge 1$ we have $\log n \le n$, hence $10n \log n \le 10n^2$; and $50 \le 50n^2$. Therefore

$f(n) = 3n^2 + 10n \log n + 50 \le 3n^2 + 10n^2 + 50n^2 = 63n^2$ $(n \ge 1)$

Constants: $c_1 = 3$, $c_2 = 63$, $n_0 = 1$: $3n^2 \le f(n) \le 63n^2$ for all $n \ge 1 \Rightarrow f(n) = \Theta(n^2)$.

f(n) is not O(n) (proof by contradiction): suppose there are $c > 0$ and n_0 with $f(n) \le c \cdot n$ for all $n \ge n_0$. Because $f(n) \ge 3n^2$, we would have $3n^2 \le c \cdot n$, i.e. $n \le c/3$, for every $n \ge n_0$. But choosing $n = \max(n_0, \lceil c/3 \rceil + 1)$ violates this. Contradiction, so $f(n) \ne O(n)$.

(g) Scalability Decision for n = 1,000,000

Operation counts use the exact formulas derived above ($\log_2 10^6 \approx 19.93$). Time = ops ÷ 10 per second. Window = 10 min = 600 s = 6,000 ops.

Module	Operations at $n = 10^6$	Running time	Meets 10-min window?
M1 (n(n−1)/2)	4.99999×10^{11}	5.0×10^{10} s ≈ 1,584 years	**No**
M2 ($2^{20} - 1$)	1,048,575 ≈ 1.05×10^6	104,858 s ≈ 29.1 hours	**No**
S2 (⌊log n⌋ + 1)	20	2 s	**Yes**
M4 current ($2n^2 - n$)	≈ 2.0×10^{12}	2.0×10^{11} s ≈ 6,338 years	**No**
M4 redesign (n log n + n)	≈ 2.09×10^7	2.09×10^6 s ≈ 24.2 days	**No** (but 95,000× faster)
M5 (n + n·log n(log n+1)/2)	≈ 2.10×10^8	2.10×10^7 s ≈ 243 days	**No**

Which modules fail? Under the stipulated rate of 10 operations/second, the budget is only 6,000 operations, so **M1, M2, M4 (both designs) and M5 fail**; only S2 passes with 20 operations (2 s). The two quadratic modules (M1, M4-current) fail catastrophically by factors of 10^8–10^9, so they are the first priority.

Recommendations

- **M1 – must be redesigned ($\Theta(n^2) \rightarrow \Theta(n)$).** Group products by the key (title, seller) using a hash table and mark every repeated key as a duplicate: expected $\Theta(n)$ time, $\Theta(n)$ space ($\approx 10^6$ operations, 500,000× fewer). Alternatively sort by (seller, title) in $\Theta(n \log n)$ and compare neighbours. Also run it incrementally on newly uploaded listings only, instead of re-scanning the whole catalogue nightly.
- **M4 – must be redesigned ($\Theta(n^2) \rightarrow \Theta(n \log n)$).** Adopt the proposed no-replication design $T(n) = 2T(n/2) + n$: about 2.1×10^7 instead of 2×10^{12} operations. Data doubling then costs ≈ 2.1× instead of 4×.
- **M5 – should be redesigned ($\Theta(n \log^2 n) \rightarrow \Theta(n \log n)$).** Re-sorting at every level is redundant. Sort once (merge sort, $\Theta(n \log n)$) or merge already-sorted halves in linear time, giving $T(n) = 2T(n/2) + n = \Theta(n \log n)$; this removes a factor of about $\log n$ (≈ 10× fewer operations here).
- **M2 – already asymptotically optimal, $\Theta(n)$.** Every request must be audited at least once, so no better asymptotic class exists. It still takes 29 hours under the stated rate, so reduce the constant: audit only new requests since the previous window (incremental or sliding-window counters), and run the audits in parallel.
- **M3 – use S2 ($\Theta(\log n)$) and retire S1 ($\Theta(n)$).** Keep the sorted index (or a hash index for exact-match lookups); S1 would need on average 750,000 comparisons at $p = 0.5$ versus at most 20.
- **Important caveat.** With only 10 operations/second, even a $\Theta(n)$ pass over 10^6 items (10^5 s ≈ 27.8 h) exceeds a 10-minute window, so after the algorithmic redesigns the platform also needs faster or parallel servers or batch/incremental processing. Asymptotic improvement is necessary but, at this stated hardware rate, not sufficient for every module.

Summary of results

Module	Operation count / recurrence	Complexity	Action
M1	$n(n-1)/2$	$\Theta(n^2)$	Redesign (hashing)
M2	$2^{\lceil \log n \rceil} - 1$	$\Theta(n)$	Keep; incremental / parallel
M3-S1	avg $[(2-p)n + p]/2$	$\Theta(n)$	Retire
M3-S2	$\log n + 1$	$\Theta(\log n)$	Keep
M4 current	$4T(n/2) + n$	$\Theta(n^2)$	Redesign
M4 redesign	$2T(n/2) + n$	$\Theta(n \log n)$	Adopt
M5	$2T(n/2) + n \log n$	$\Theta(n \log^2 n)$	Sort once → $\Theta(n \log n)$
Report generator	$3n^2 + 10n \log n + 50$	$\Theta(n^2)$	Remove quadratic step